

HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

# Distributed Computing

Michael Kosyakov
Associate Professor

Denis Tarakanov
Assistant Lecturer

ifmo.distributedclass.bot@gmail.com

# Литература




Введение в распределенные вычисления. Учебное пособие.

Косяков М.С. СПб: НИУ ИТМО, 2014 – 155с.

… и литература из списка в пособии

# What is this course about?

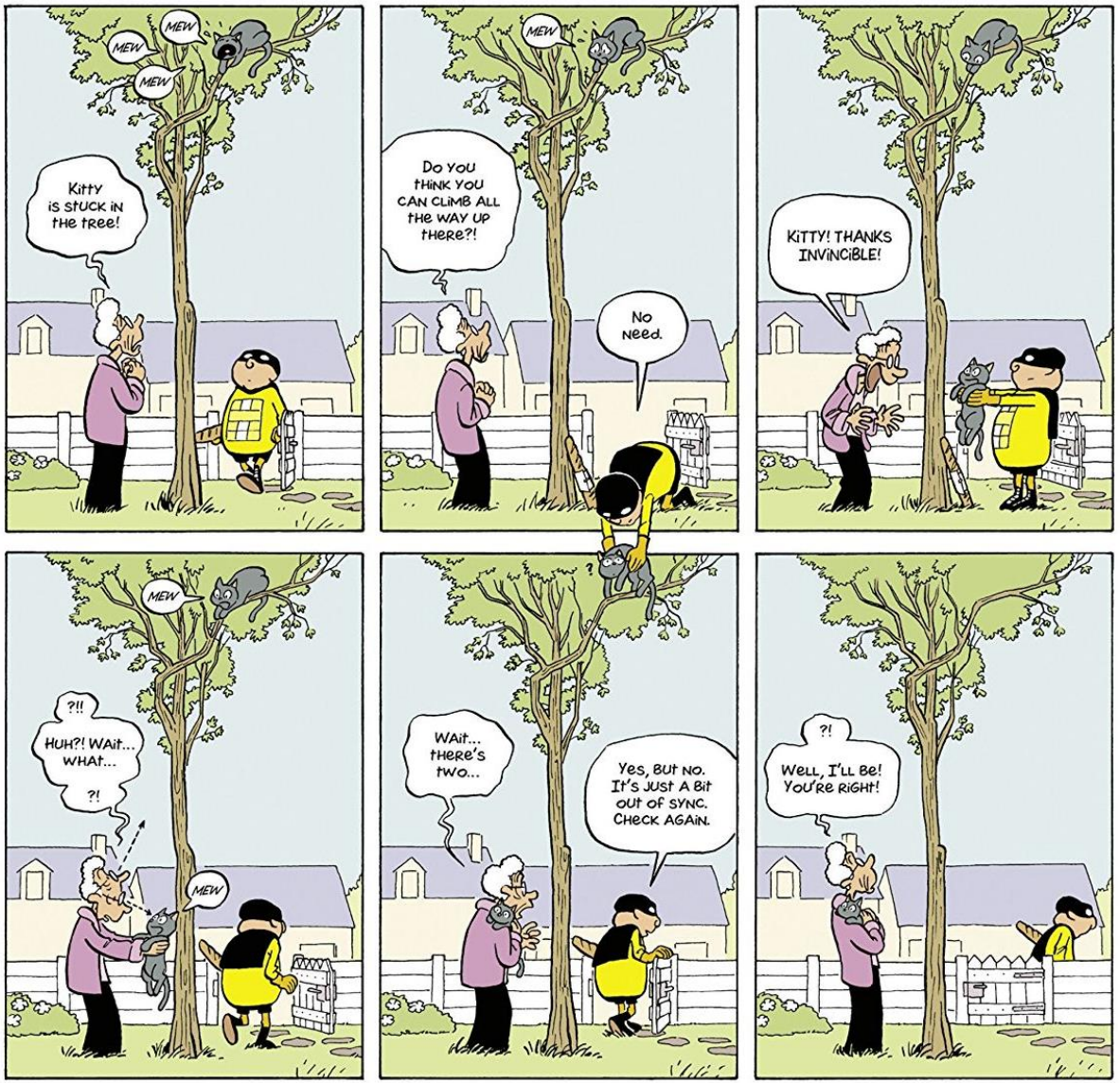




- This course is NOT about
  - Distributed systems
  - Network protocols
  - Network programming
- “The field of distributed computing covers all aspects of computing and information access across multiple processing elements connected by any form of communication network, whether local or wide-area in the coverage” (DC: Principles, Algorithms and Systems)

# What is this course about?





# Synchronization

- In the everyday sense synchronization means the act of making things happen at the same time
- In computer systems synchronization refers to relationships among events: before, at the same time, after
- Ordering requirements
  - Serialization: Event A must happen before event B
  - Mutual exclusion: Events A and B must not happen at the same time
- So we will study various techniques for enforcing ordering requirements in distributed environment

# Synchronization

- How do we check and satisfy ordering requirements in real life? E.g. who had lunch first: Alice or Bob living in different cities?
  - Usually we just compare the times of the events
- In computer systems often we need to satisfy ordering requirements without the benefit of a clock
  - No common universal clock
  - No chance to record the exact time of everything
- How do we guarantee that Alice will eat lunch before Bob?

# Serialization with messages



HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

| Alice (Thread A) | Bob (Thread B) |
|---|---|
| 1. Eat breakfast<br>2. Work<br>3. Eat lunch<br>4. Call Bob | 1. Eat breakfast<br>2. Wait for a call<br>3. Eat lunch |

- Alice and Bob ate lunch **sequentially**, but ate breakfast **concurrently**
- Two events are concurrent if we cannot tell by looking at the program code which will happen first

# Communication via shared variables





- Shared variables are just means for threads to interact with each other
  - One thread writes a value to be read by another thread
- But will the reader see the new value written by the writer or an old value that was already there?
  - The thread are unsynchronized if we can't assert it by looking at the program code
- Often we would like to enforce the reader to not read until after the writer writes
- This is exactly the same serialization problem …





# Concurrent updates

- Shared variable `count` being updated by two threads: `count++;`
  - Is increment atomic?
  - Assuming the initial value is zero, what could be the result?
- Two threads run the following code concurrently: `for (i=0; i<100; ++i) count++;`
  - What are the largest and the smallest possible values, assuming that each increment fetches data from memory and stores updated value back?
- Mutual exclusion guarantees that only one thread accesses a shared variable at a time





# Course topics

- Subject of distributed computing
- A model of distributed computations
- Casual order of events and Logical Clocks
- Distributed mutual exclusion algorithms
- Tracking causality with Vector Clocks

# Subject of distributed computing

# Definitions of distributed system

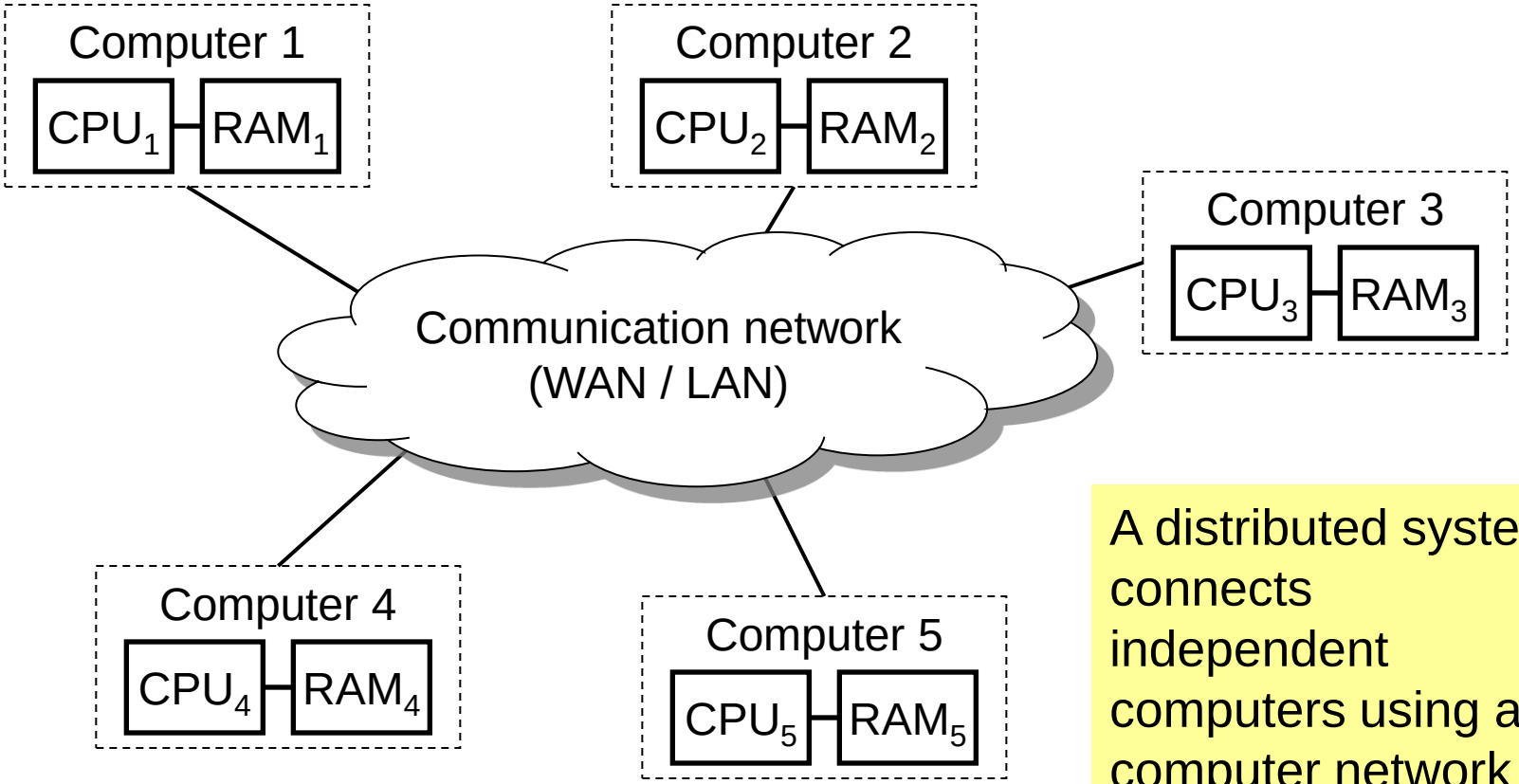




- You know you are using one when the crash of a computer you have never heard of prevents you from doing work (Leslie Lamport)
- A collection of independent computers that appears to the users of the system as a single coherent computer (Andrew Tanenbaum)
- A collection of computers that do not share common memory or a common physical clock, that communicate by a messages passing over a communication network, and where each computer has its own memory and runs its own operating system (Singhal-Shivaratri)

# Definitions of distributed system






A distributed system connects independent computers using a computer network

# Relation to multiprocessors





Stop! But why did we talk about concurrent programs running on multiprocessor machines?

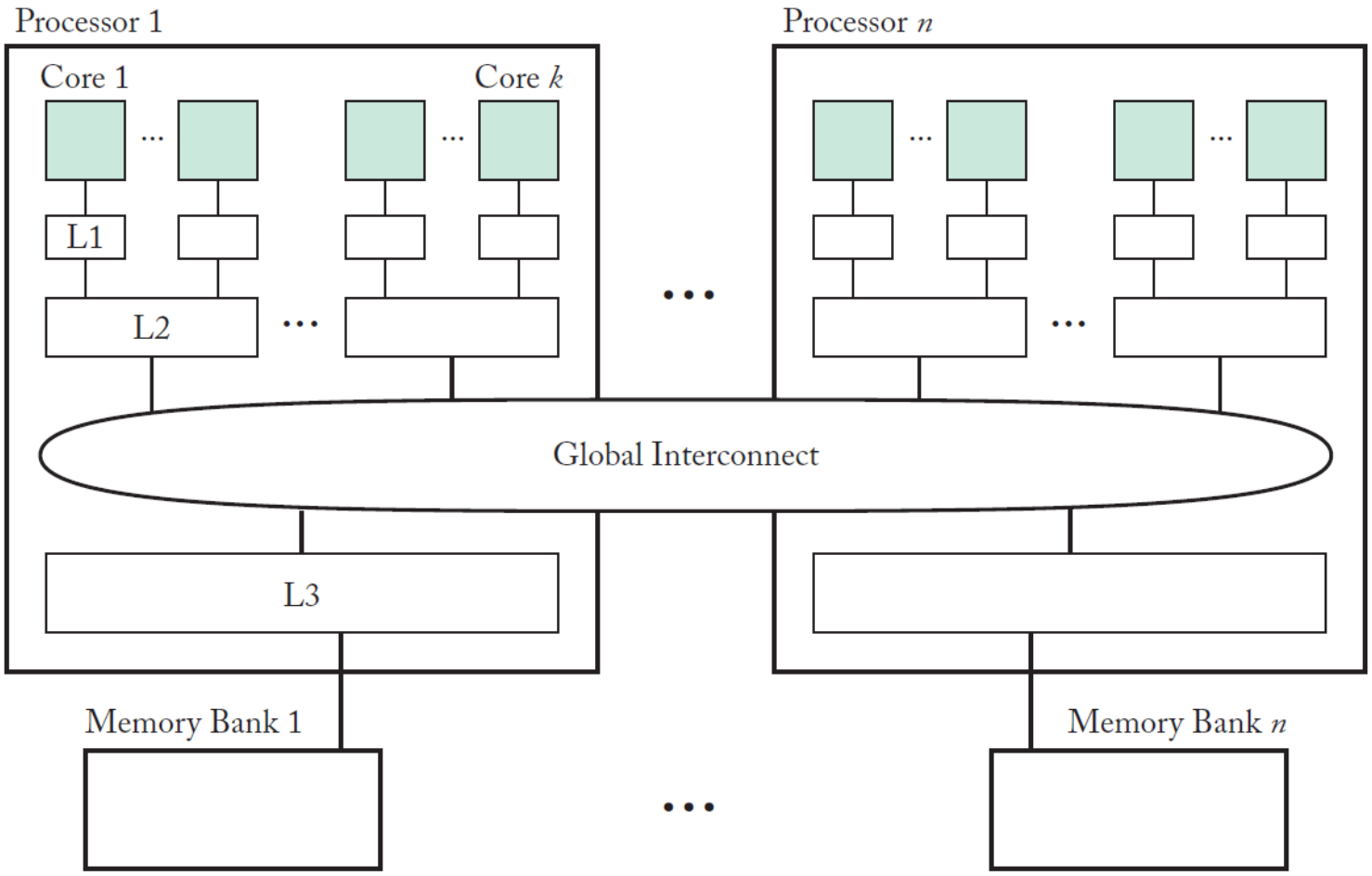
«The network is the computer»

John Gage, Sun Microsystems

«… but the computer is also the network»

# Relation to multiprocessors






Typical nonuniform memory access (NUMA) machine. Numbers of components of various kinds, and degree of sharing at various levels, differs across manufacturers and models.

# Interacting processes





- All of the activity in distributed system is performed by interacting processes
- Each process has its own state, consisting of the set of data it can access and update
- The state of each process is completely private, it can't be accessed and updated by any other process
- Processes interact by passing messages to exchange data and coordinate their activity
- Process can only know state information via received messages, which represent the state of the remote process sometime in the past
- Accuracy with which processes can be coordinated is limited by communication delays and by the difficulty of maintaining the same notion of time

# Communication delays





**Table 2.2** Example Time Scale of System Latencies

| Event | Latency | Scaled |
|---|---|---|
| 1 CPU cycle | 0.3 ns | 1 s |
| Level 1 cache access | 0.9 ns | 3 s |
| Level 2 cache access | 2.8 ns | 9 s |
| Level 3 cache access | 12.9 ns | 43 s |
| Main memory access (DRAM, from CPU) | 120 ns | 6 min |
| Solid-state disk I/O (flash memory) | 50–150 µs | 2–6 days |
| Rotational disk I/O | 1–10 ms | 1–12 months |
| Internet: San Francisco to New York | 40 ms | 4 years |
| Internet: San Francisco to United Kingdom | 81 ms | 8 years |
| Internet: San Francisco to Australia | 183 ms | 19 years |
| TCP packet retransmit | 1–3 s | 105–317 years |
| OS virtualization system reboot | 4 s | 423 years |
| SCSI command time-out | 30 s | 3 millennia |
| Hardware (HW) virtualization system reboot | 40 s | 4 millennia |
| Physical system reboot | 5 m | 32 millennia |

# Computer clocks and event timing





- Computer clocks drift from perfect time, and drift rates differ from one another
- Ordinary quartz-based clocks have drift rate ~ 1 usec per sec, giving a difference of 86 msec every day
- But what about physical clock synchronization?
  - There is always a bound on how accurately we can synchronize clocks affected by variable message delays
  - E.g. if there is an upper *max* and lower *min* bounds on time taken to transmit any message, then optimum bound that can be achieved on clock skew when synchronizing *N* clocks is $u\ (1 - 1/N)$, where $u = max - min$ is uncertainty in message transmission time

# Computer clocks and event timing

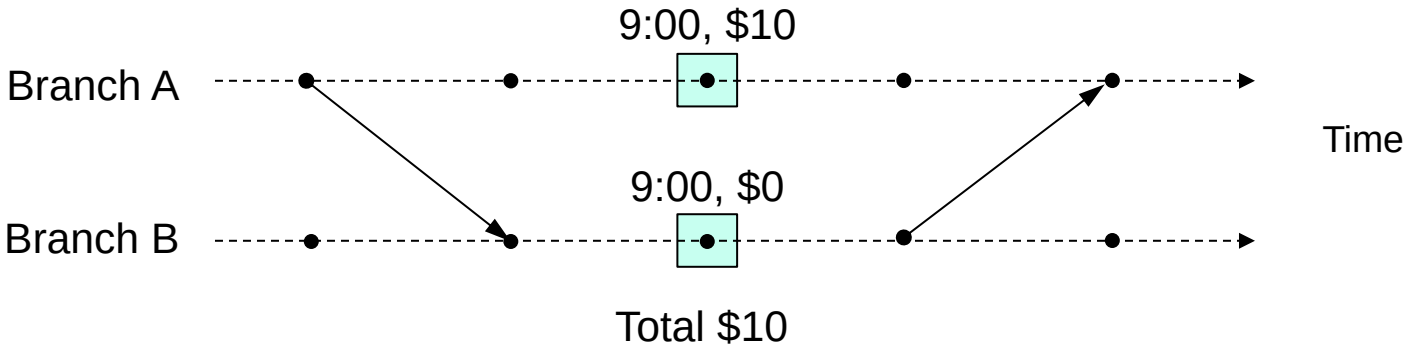


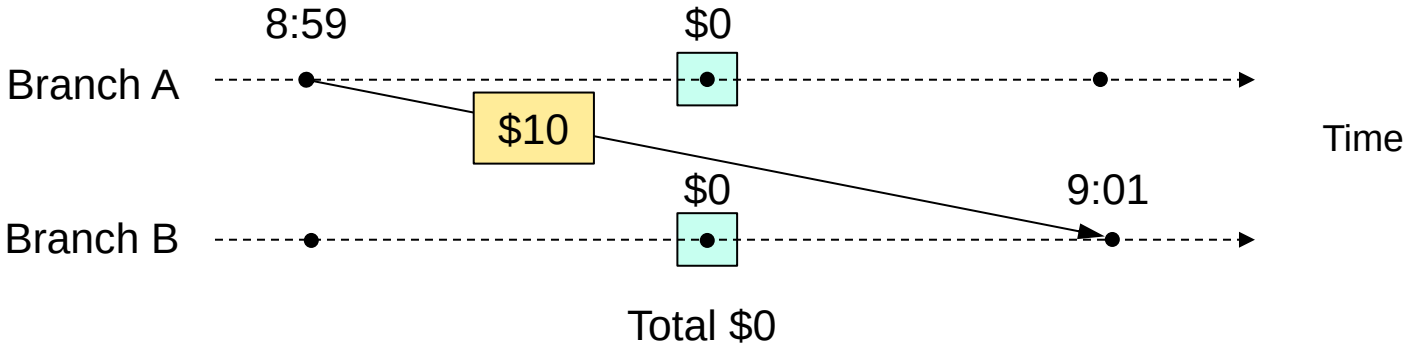


- So processes running on different machines may read different time values even if read at the same time instant
- Applications that expect that timestamp associated with an event is correct and does not depend on the computer where it was registered (that is, the clock that was used) may not work correctly
- Let's consider banking system as an example…

# Banking system

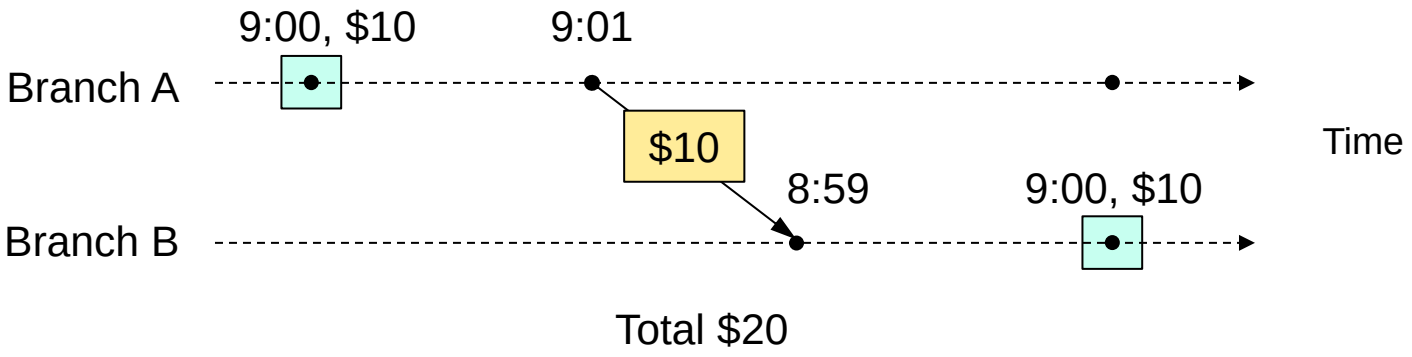




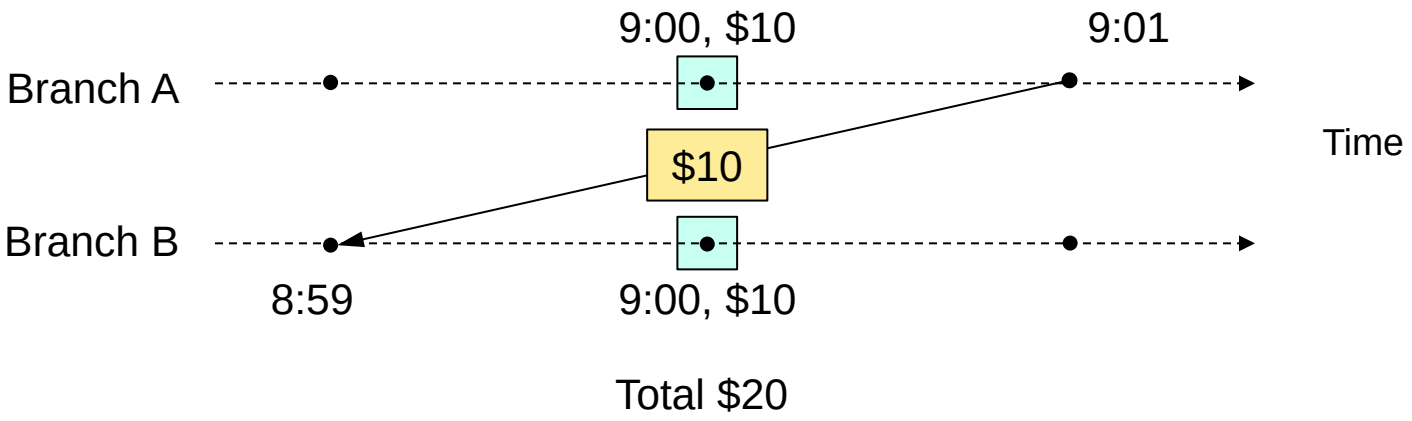
# Banking system





# Banking system






From 8:59 to 9:01 Grandma sees two “cats”

# Degree of synchronization





- Synchronous clocks: in a system with synchronous clocks all physical clocks show the same time
- But we said that clocks can never be perfectly synchronized …
- Hence, a weaker notion of synchronized clocks is that drift rate of local clocks from real time has a known upper bound
- Synchronous processes: synchronous processes take actions in lockstep synchrony, i.e. in each step each process execute its action
- But in reality there can be arbitrary delay between executions of two consecutive instructions …
- Hence, a weaker notion is that time to execute each step of a process has known lower and upper bounds

# Degree of synchronization





- Synchronous channels: a channel is called synchronous, when there is a known upper bound on message transmission delay along that channel
- Synchronous communication: a sender sends a message only when the receiver is ready to receive it, and vice versa
- In asynchronous communication there is no coordination between the sender and the receiver
- Synchronous systems should guarantee all stated bounds! … Otherwise, any design based on that values will not be reliable
- In real life systems are often asynchronous because processes need to share the processors and communication channels need to share the network

# Event ordering

There are only two hard problems in distributed systems:

2. Exactly-once delivery

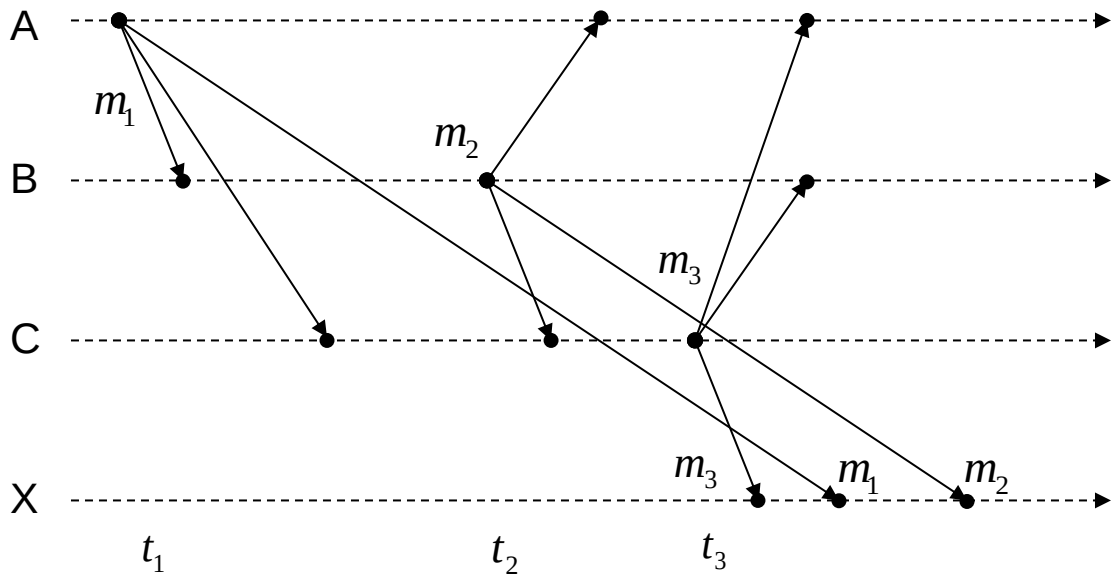
1. Guaranteed order of messages

2. Exactly-once delivery

-- Mathias Verraes

# Event ordering

Often we are interested just in event order, not in particular time events happen




not necessarily
t1 < t2 < t3

Different users may observe messages in different order

# Event ordering

Initially shared variables flag1 == flag2 == 0

| Thread 1 | Thread 2 |
| --- | --- |
| if ( flag2 == 0 ) {<br>  flag1 = 1;<br>  // critical section<br>} else<br>  // resolve conflict | if ( flag1 == 0 ) {<br>  flag2 = 1;<br>  // critical section<br>} else<br>  // resolve conflict |

Can both threads run simultaneously inside critical section?





# Event ordering

Initially shared variables flag1 == flag2 == 0

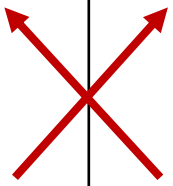
| Thread 1 | Thread 2 |
|---|---|
| flag1 = 1;<br>if ( flag2 == 0 ) {<br>  // critical section<br>} else<br>  // resolve conflict | flag2 = 1;<br>if ( flag1 == 0 ) {<br>  // critical section<br>} else<br>  // resolve conflict |

Can both threads run simultaneously inside critical section?

# Event ordering

Initially shared variables flag1 == flag2 == 0

| Thread 1 | Thread 2 |
|---|---|
| … | … |
| flag1 = 1; // $a_1$ | flag2 = 1; // $b_1$ |
| if ( flag2 == 0 ) { // $a_2$ | if ( flag1 == 0 ) { // $b_2$ |
| // critical section | // critical section |
| } else | } else |
| // resolve conflict | // resolve conflict |

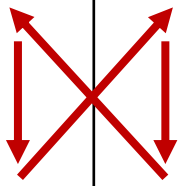


- If T1 is in CS then $a_2$ should happen before $b_1$: $a_2 \rightarrow b_1$
- If T2 is in CS then $b_2$ should happen before $a_1$: $b_2 \rightarrow a_1$

# Event ordering

Initially shared variables flag1 == flag2 == 0

| Thread 1 | Thread 2 |
|---|---|
| … | … |
| flag1 = 1; // $a_1$ | flag2 = 1; // $b_1$ |
| if ( flag2 == 0 ) { // $a_2$ | if ( flag1 == 0 ) { // $b_2$ |
| // critical section | // critical section |
| } else | } else |
| // resolve conflict | // resolve conflict |



If both threads are in CS then combining $a_2 \rightarrow b_1$ and $b_2 \rightarrow a_1$ with program order $a_1 \rightarrow a_2$ and $b_1 \rightarrow b_2$ we get an apparent ordering loop, which “should” be impossible





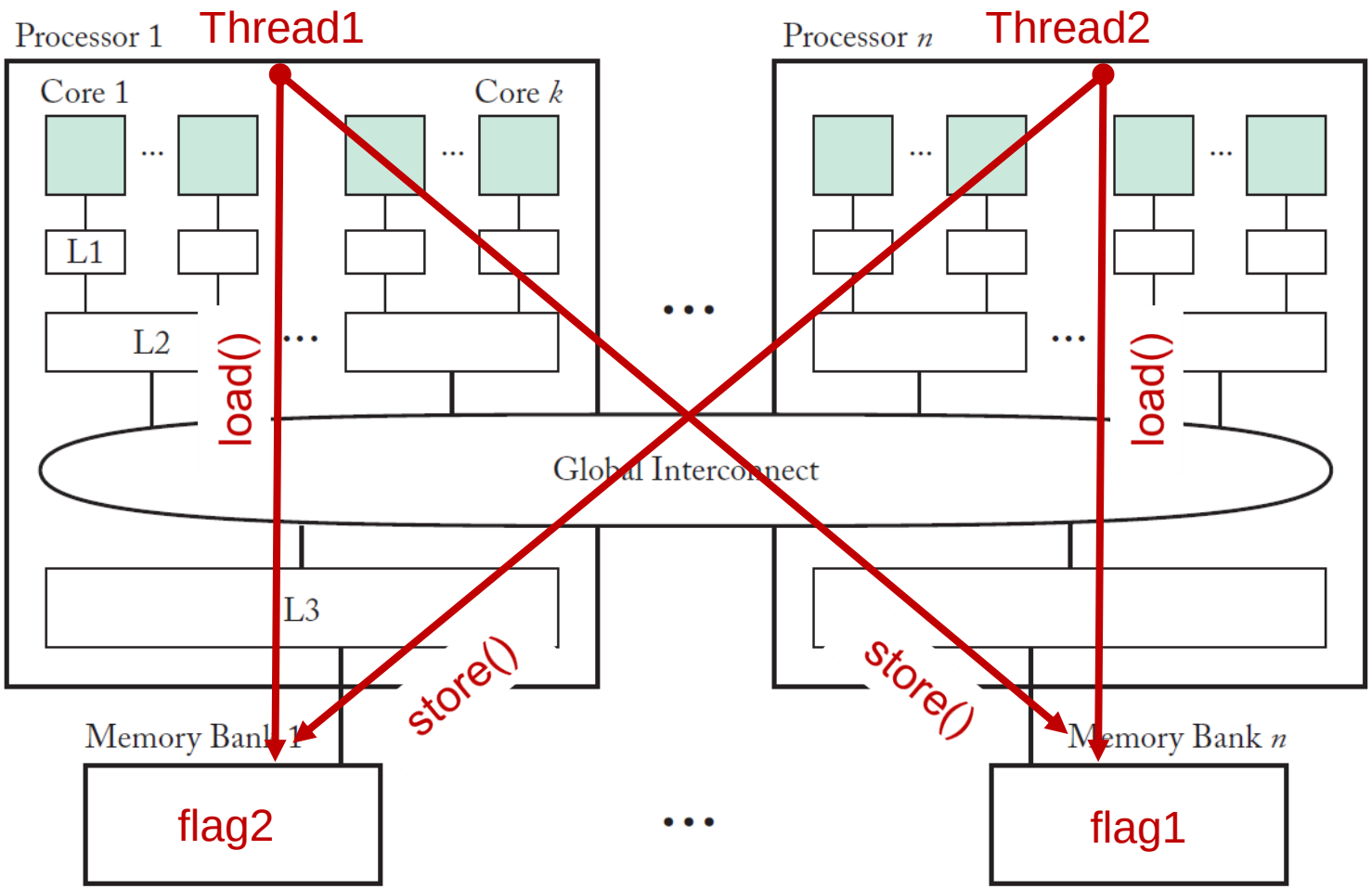
# Event ordering

Initially shared variables flag1 == flag2 == 0

| Thread 1 | Thread 2 |
|---|---|
| … | … |
| flag1 = 1; // $a_1$ | flag2 = 1; // $b_1$ |
| if ( flag2 == 0 ) { // $a_2$ | if ( flag1 == 0 ) { // $b_2$ |
| // critical section | // critical section |
| } else | } else |
| // resolve conflict | // resolve conflict |

- Do flag1 and flag2 memory locations already contain new value?
- Memory fences are necessary

# Event ordering




Modern RAM is not sequentially consistent, i.e.
there is no “global order” of all memory accesses





# Event ordering

Initially A == B == 0

| Thread 1 | Thread 2 | Thread 3 |
|---|---|---|
| A = 1; | if (A == 1)<br>B = 1; | if (B == 1)<br>X = A; |

Can X get value 0 provided that B == 1 in T3?





# Event ordering

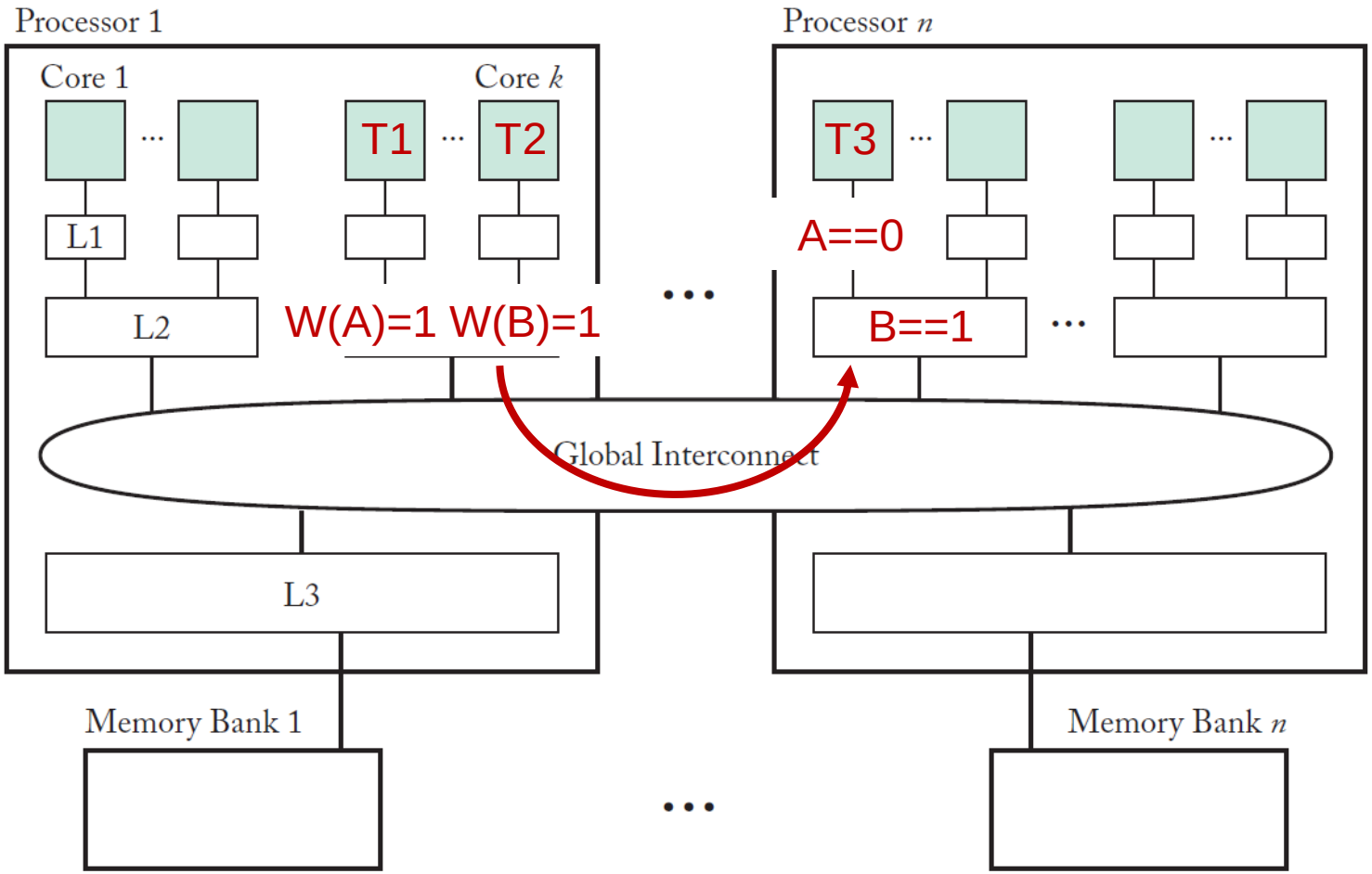
Initially A == B == 0

| Thread 1 | Thread 2 | Thread 3 |
|---|---|---|
| A = 1; | if (A == 1)<br>B = 1; | if (B == 1)<br>X = A; |

Is Write(A) in T1 seen atomically by all threads at the same time?

T1's Write(A) is seen by T2, but not by T3, while T2's Write(B) is seen by T3, allowing X to get value 0

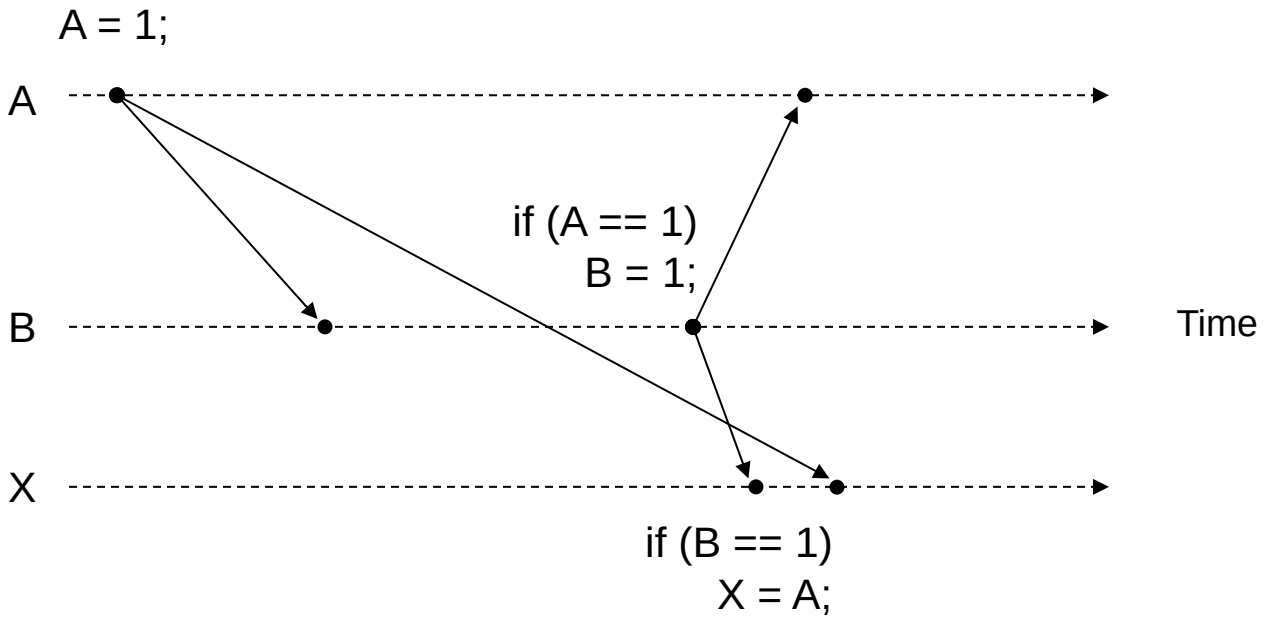
# Event ordering




On a multiprocessor data in upper levels of the memory hierarchy may not be up-to-date if it's been modified by some thread on another core
Propagating changes to multiple caches is inherently a nonatomic operation

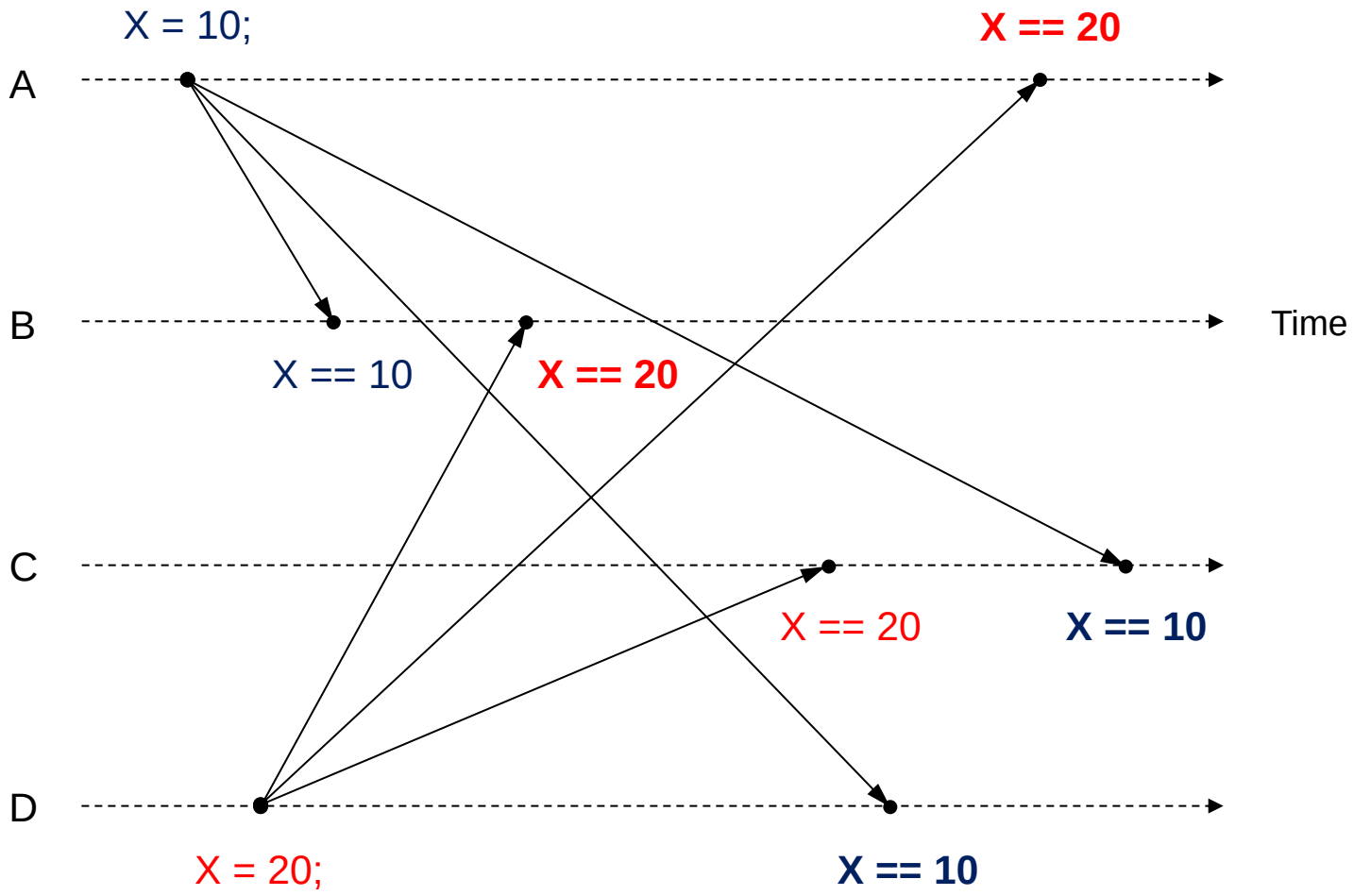
# Event ordering

"The ARM architecture is not multicopy atomic: a write by one hardware thread can become visible to some other threads before becoming visible to all of them. Microarchitecturally, this behaviour can arise from multi-level cache and core clustering. Two cores that share a cache level that is not shared with a third core can observe each other's memory accesses before the third core does."




Different users may observe messages in different order

# Cache coherence







# Cache coherence

Initially X == flag1 == flag2 == 0

| Thread 1 | Thread 2 | Thread 3 | Thread4 |
|---|---|---|---|
| X = 10;<br>flag1 = 1; | X = 20;<br>flag2 = 1; | while(flag1 != 1);<br>while(flag2 != 1);<br><br>r1 = X; | while(flag1 != 1);<br>while(flag2 != 1);<br><br>r2 = X; |

Can r1 get value 10 and r2 get value 20?

# Cache coherence

Initially X == flag1 == flag2 == 0

| Thread 1 | Thread 2 | Thread 3 | Thread4 |
|---|---|---|---|
| X = 10;<br>flag1 = 1; | X = 20;<br>flag2 = 1; | while(flag1 != 1);<br>while(flag2 != 1);<br><br>r1 = X; | while(flag1 != 1);<br>while(flag2 != 1);<br><br>r2 = X; |

Can r1 get value 10 and r2 get value 20?

Cache coherence: changes to the same memory location are seen in the same order by all threads (by maintaining at-most-one writable copy of any given cache line)

# Fallacies of distributed computing





More than 20 years ago Peter Deutsch and James Gosling defined the 8 fallacies of distributed computing:

“Essentially everyone, when they first build a distributed application, makes the following eight assumptions. All prove to be false in the long run and all cause big trouble and painful learning experiences.

1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. The network is secure
5. Topology doesn’t change
6. There is one administrator
7. Transport cost is zero
8. The network is homogeneous”